DOCUMENTATION

1. Overall Summary

Project Identity

- **Project Title:** Dr. Jay Deep Ghosh – Official Medical Portfolio & Patient Portal
- **URL:** `https://kapil-97.github.io/`
- **Developer:** Kapil Krishna Sarma
- **Client:** Dr. Jay Deep Ghosh (Senior Consultant, Orthopaedics & Spine Surgery, Apollo Hospitals, Guwahati)

1.1 Project Abstract

I designed this comprehensive full-stack web application to serve as the digital headquarters for Dr. Jay Deep Ghosh. My goal was to create a platform that serves a dual purpose: establishing digital authority through informational dissemination and providing functional utility by facilitating patient intake.

The website bridges the gap between the doctor and patients in Northeast India. By providing detailed medical information, showcasing successful surgical outcomes, and offering a streamlined, digital-first appointment booking system, I have replaced traditional, fragmented communication channels with a centralized, professional hub.

1.2 Core Objectives

- **Digital Presence & Branding:** My primary objective was to establish a professional online identity for Dr. Ghosh independent of hospital directories, specifically highlighting his expertise in Minimally Invasive Spine Surgery (MISS) and complex orthopaedics.
- **Patient Education:** I aimed to demystify complex medical procedures (e.g., Endoscopic Spine Surgery, TKR) through accessible content, educational videos, and detailed "Before & After" case studies.
- **Operational Efficiency:** To reduce administrative burden, I automated the initial patient intake process. The secure, serverless appointment booking system collects patient data, symptoms, and medical reports before the consultation even begins.

1.3 Architectural Overview

I utilized a **Decoupled Architecture** to ensure the system is both scalable and easy to maintain:

- **Frontend:** I built the User Interface with **HTML5, CSS3, and JavaScript**, prioritizing a mobile-first design philosophy to cater to the high volume of mobile internet users in the region.

- **Backend:** I architected a serverless infrastructure using **Supabase**. This handles data persistence (PostgreSQL), file storage (medical reports), and business logic (Edge Functions) without the overhead of managing a traditional VPS.

1.4 Key Functional Modules

- **Home & About:** A dynamic landing page featuring key performance metrics (Years of Experience, Surgery Counts), a biography, and professional credentials.
- **Clinical Procedures:** A catalog of services detailing surgical interventions like Scoliosis Correction and Joint Replacements, aiding in patient decision-making.
- **Patient Outcomes & Testimonials:** A multimedia gallery featuring "Before & After" radiological evidence and patient video testimonials to build social proof.
- **Appointment System:** A robust form designed to capture Patient Demographics, Visit Type, and Medical Report Uploads. I also integrated a custom CAPTCHA to prevent spam.
- **Admin Dashboard (Internal):** A restricted-access interface I built for the doctor and staff to view incoming appointments and manage patient queries.

---

2. Frontend Documentation

2.1 HTML Structure & Architecture

2.1.1 Global Configuration (The `<head>` Strategy)

I configured every page with a unified block designed for Performance, SEO, and Social Sharing to ensure a consistent user experience.

Tag / Element	Attribute Configuration	Technical Purpose
Meta Viewport	`width=device-width, initial-scale=1`	**Responsive Design Trigger:** Forces the browser to render pixels at a 1:1 ratio with the device screen, preventing mobile browsers from zooming out by default.
Theme Color	`content="#05668d"`	**Browser UI Integration:** Changes the color of the mobile Chrome/Brave address bar to match the brand's primary blue, creating a native app-like feel.
SEO Meta	`name="robots" content="index, follow"`	**Crawler Directives:** Explicitly allows Google/Bing bots to index the page and follow links to other pages within the site.
Open Graph	`property="og:image"`	**Social Preview:** Defines the specific thumbnail image displayed when the

		link is shared on WhatsApp, Facebook, or LinkedIn.
Preconnect	`rel="preconnect" href="https://fonts.gstatic.com"`	**Performance:** Initiates a network handshake with Google's font server before the CSS requests the font, reducing "Flash of Unstyled Text" (FOUT).
Favicon	`rel="icon" type="image/png"`	**Branding:** Displays the logo in the browser tab.

2.1.2 Landing Page Architecture (`index.html`)

Type: Marketing / Conversion Landing Page

Key Logic: I combined high-impact visuals with trust signals to drive conversions.

A. Hero Slider (`.swiper-wrapper`)

I implemented a nested structure to support the Swiper.js library. The `slide-image-wrapper` uses absolute positioning to sit behind the text content, ensuring the text remains readable regardless of the image dimensions.

B. Dynamic Statistics (`.stats-grid`)

This section utilizes data attributes to drive the JavaScript animation. I used the `data-target` attribute to store the final number (e.g., "23"). The JavaScript reads this, starts the counter at 0, and increments it until it hits the target, creating an engaging visual effect.

C. Image Modals (`#image-modal`)

I placed a hidden DOM element at the bottom of the `<body>` to act as a global modal. By default, it is hidden (`aria-hidden="true"`). When a user clicks a gallery image, a simple JS function swaps the `src` of the modal image and makes it visible, avoiding the need for heavy lightbox libraries.

2.1.3 Functional Modules

A. Appointment System (`appointment.html`)

Type: Data Collection Interface

Key Logic: I designed this for secure form handling without a backend server.

Input Field	HTML5 Type	Validation Logic
`honeypot`	`text`	**Anti-Spam:** I hid this input via CSS. Real users don't see it, but bots will fill it. If the backend receives data here, I reject the form immediately.
`phone`	`tel`	**UX:** Triggers the numeric keypad on mobile devices for easier entry.
`date`	`text`	**UI:** I connected this to the Flatpickr JS library to ensure a consistent calendar interface across all browsers.
`file`	`file`	**Security:** `accept="image/*,.pdf"` restricts uploads to safe formats only.

The CAPTCHA Mechanism:

I implemented a custom math challenge to prevent bot spam without using invasive Google Recaptcha. The JavaScript generates a simple math question (e.g., "5 + 3"), and the form submission is blocked unless the user's input matches the calculated answer.

B. Admin Dashboard (`admin.html`)

Type: Single Page Application (SPA)

Key Logic: Authentication and View Switching.

I built this file as a unique Single Page Application containing two distinct views:

1. **Authentication (`#login-section`):** Visible by default. On a successful login event, I hide this section.
2. **Dashboard (`#dashboard-section`):** Hidden by default. Once authenticated, the JavaScript injects patient data into the `#appointment-list` table rows and enables the status filters.

2.1.4 Content Pages Structure

A. About Profile (`about.html`)

I used a custom Accordion Component for the Education and Awards sections. The panel acts as a sibling to the trigger button. When clicked, the JavaScript toggles a `max-height` style on the panel to animate the opening and closing seamlessly.

B. Video Gallery (`outcomes.html` & `videos.html`)

Performance Strategy: To prevent the page from crashing on mobile devices due to multiple YouTube iframes, I implemented a **"Facade Pattern."**

Instead of embedding the videos directly, I display a lightweight thumbnail image with a "Play" button. The actual YouTube `<iframe>` is only injected into the modal when the user explicitly clicks the thumbnail. This significantly improves the Initial Page Load speed.

2.1.5 Accessibility & Semantic Roadmap

I wrote the HTML to strictly follow WCAG 2.1 standards:

- **Landmarks:** I used semantic tags like `<header>`, `<main>`, `<aside>`, and `<footer>` to allow screen readers to navigate the page structure easily.
- **ARIA Labels:** I added `aria-label="Close modal"` to buttons that only contain icons, ensuring visually impaired users know what the button does.
- **Alt Text:** Every image tag includes a descriptive `alt` attribute (e.g., "X-ray showing scoliosis correction") rather than generic filenames.

2.2 CSS Documentation

This section provides a technical breakdown of the stylesheets. I utilized a distributed CSS architecture, categorizing styles by function (Base, Layout, Components, Sections) to make maintenance easier.

2.2.1 `base.css` (Variables & Reset)

Purpose: Defines the global design tokens (variables) and normalizes browser inconsistencies.

A. CSS Variables (`:root`)

I established a centralized design system using CSS Custom Properties.

Variable Name	Value	Description / Usage
`--color-primary-blue`	`#05668d`	The dominant brand color, used for headers and primary actions.
`--color-primary-green`	`#00a896`	The accent color, used for hover states and icons.
`--brand-gradient`	`linear-gradient(...)`	A pre-defined blue-green gradient used on primary buttons.
`--font-heading`	`'Montserrat'`	A bold sans-serif font for authority (headings).
`--font-body`	`'Inter'`	A highly legible font for reading text.
`--container`	`1120px`	The maximum width constraint for the main content.

B. Global Resets & Typography Rules

- **`* { box-sizing: border-box; }`**: I applied this to ensure padding and borders are included in an element's total width, preventing layout breakage.

- **`html { scroll-behavior: smooth; }`**: This enables a smooth scrolling effect when users click anchor links.
- **Fluid Typography:** I used fluid units (`var(--step-4)`) for headings so they scale naturally between mobile and desktop screens. For paragraphs, I set a `max-width: 75ch` to ensure lines don't become too long to read comfortably.

2.2.2 `layout.css` (Grid & Structure)

Purpose: Manages the macro-layout (Header, Footer, Grids) and responsive behavior.

A. The Grid System

I created utility classes like `.grid-3` to handle layouts efficiently.

- **Desktop:** `grid-template-columns: repeat(3, 1fr)` creates a 3-column layout.
- **Mobile:** I added a media query that changes this to `1fr`, causing elements to stack vertically on smaller screens automatically.

B. Header & Navigation (`.site-header`)

I positioned the header as `fixed` with a high `z-index` to keep it visible while scrolling. To give it a modern feel, I added a "Glassmorphism" effect using `backdrop-filter: blur(10px)`.

2.2.3 `components.css` (UI Library)

Purpose: Contains reusable "atomic" design elements.

A. Button Components (`.btn`)

I defined a base `.btn` class with a pill shape (`border-radius: 999px`) and smooth transitions. The `.btn-primary` modifier applies the brand gradient and a lift effect (`transform: translateY(-2px)`) on hover to encourage clicks.

B. Card Component (`.card`)

Used for Services and Testimonials, these cards have a white background, subtle border, and overflow handling to ensure images stay within the rounded corners.

2.2.4 `sections.css` (Page Specifics)

Purpose: Handles unique styling for specific page sections.

A. Hero Section (`.hero`)

I set the hero height to `92vh` to cover most of the viewport. The background defaults to black as a fallback, and I used `object-fit: cover` on the images to ensure they fill the screen without distortion.

B. Admin Dashboard Internal CSS

For the Admin panel, I used an isolated style block. This includes specific status badges (Green for confirmed, Orange for pending) and a dark sidebar design (`#1e293b`) to distinguish the internal tool from the public website.

2.3 JavaScript Logic & Backend Integration

I wrote the application using **Vanilla ES6+ JavaScript** to ensure maximum performance without the overhead of heavy frameworks like React. The code is encapsulated within Immediately Invoked Function Expressions (IIFE) to prevent global variable pollution.

2.3.1 Core Application Logic (`main.js`)

Role: The "Central Nervous System" of the website. It handles backend connections, global UI interactions, and form submissions.

A. Supabase Configuration (Backend Handshake)

Instead of bundling a heavy SDK, I implemented a **Lazy Loading Pattern**. The Supabase library is only fetched when the user actually attempts an interaction that requires it (like submitting a form), reducing the initial page load time by ~150kb.

B. Toast Notification System (`showToast`)

I built a custom notification engine to replace standard browser alerts. It dynamically creates a DOM element centered on the screen, applies a Green (Success) or Red (Error) border based on the outcome, and automatically dismisses itself after 4 seconds to keep the UI clean.

C. Contact Form & Backend Integration (`initContactForm`)

This function connects the HTML `<form>` to the Supabase Database.

- **Honeypot Security:** I check the hidden `honeypot` field. If a bot has filled it, I abort the database call immediately but show a fake "Success" message to confuse the bot.
- **Data Serialization:** I use `FormData` to extract user inputs and map them directly to the Supabase columns (`name`, `phone`, `email`, `message`).
- **The API Call:** The script executes `await supabase.from('inquiries').insert([inquiryData])`. This is the critical line that pushes data to the cloud.

D. UI Interaction Modules

- **Sticky Header:** I used `requestAnimationFrame` to throttle scroll events, ensuring the header background toggle doesn't cause browser lag.
- **Video Facades:** I wrote logic to detect if a video is vertical (Shorts) or horizontal, calculate the correct aspect ratio, and inject the iframe only upon user interaction.

2.3.2 Accordion System (`accordion.js`)

Role: Manages expandable content areas.

Smooth Height Animation Strategy:

Since browsers cannot animate height from `0` to `auto`, I wrote a script that calculates the exact pixel height of the hidden content (`panel.scrollHeight`) and explicitly sets it. Once the transition ends, I reset it to `auto` so the text block can adapt naturally if the window resizes.

2.3.3 Custom Touch Slider (`swiper.js`)

Role: A lightweight carousel engine.

Pointer Event Logic:

Instead of relying on simple clicks, I used **Pointer Events** to unify mouse and touch interactions. The script captures the start and end coordinates of a swipe. If the movement exceeds a threshold (80px), it triggers a slide change. I also added a check to ensure that clicking a "Book Appointment" button inside the slider doesn't accidentally trigger a drag event.

---

3. Backend & Cloud Infrastructure

3.1 Architectural Overview

I designed the backend using a **Serverless, Event-Driven Architecture** powered by Supabase. This approach allowed me to decouple the frontend from the logic, ensuring high availability and security without needing to manage a traditional Virtual Private Server (VPS).

- **Database Engine:** PostgreSQL 15 (Managed by Supabase).
- **Serverless Runtime:** Deno (Edge Functions).
- **Infrastructure as Code (IaC):** Defined via TOML configuration.

3.2 Database Schema & Data Modeling

I utilized a normalized relational schema with two primary tables to segregate strict clinical data from general website inquiries.

3.2.1 Table: `appointments`

Purpose: Stores High-Sensitivity Patient Health Information (PHI).

Triggers: INSERT events on this table trigger the `resend-email` Edge Function.

Column Name	Data Type	Constraints	Description
`id`	`UUID`	`PK`, `Default: gen_random_uuid()`	Unique Record ID.
`created_at`	`TIMESTAMPTZ`	`Default: now()`	Precise timestamp of submission.
`patient_name`	`TEXT`	`NOT NULL`	Full legal name of the patient.
`phone`	`TEXT`	`NOT NULL`	Contact number (stored as text to retain country codes).
`email`	`TEXT`	`NULLABLE`	Optional contact method.
`preferred_date`	`DATE`	`NOT NULL`	The requested consultation date.
`visit_type`	`TEXT`	`NOT NULL`	Enum: 'First Consultation', 'Follow-up', 'Second Opinion'.
`symptoms`	`TEXT`	`NULLABLE`	Patient's self-reported clinical reason for visit.
`location`	`TEXT`	`NULLABLE`	City/Town (Demographic data).
`file_url`	`TEXT`	`NULLABLE`	Direct URL to the X-Ray/MRI in the Storage Bucket.
`status`	`TEXT`	`Default: 'Pending'`	Workflow State: `Pending` $\to$ `Confirmed` $\to$ `Completed`.

3.2.2 Table: `inquiries`

Purpose: Stores low-sensitivity contact form messages.

Column Name	Data Type	Constraints	Description
`id`	`UUID`	`PK`	Unique Record ID.
`created_at`	`TIMESTAMPTZ`	`Default: now()`	Timestamp of inquiry.
`name`	`TEXT`	`NOT NULL`	Sender Name.
`phone`	`TEXT`	`NOT NULL`	Sender Phone.
`email`	`TEXT`	`NULLABLE`	Sender Email.
`message`	`TEXT`	`NOT NULL`	Inquiry body text.

status	TEXT	Default: 'Pending'	Read/Unread status.

3.3 Security & Access Control (Row Level Security)

I implemented a "Zero Trust" model. Direct access to tables is blocked; all access is mediated through **Row Level Security (RLS)** policies.

3.3.1 Public Access Policy (The "Drop-Box" Model)

- **Target Role:** `anon` (Unauthenticated / Public Users).
- **Permitted Operation:** `INSERT` only.
- **Security Outcome:** I configured the policy so that a public user can *submit* data but cannot query, read, or modify existing records. If a hacker attempts a `SELECT * FROM appointments`, the database returns 0 rows.

3.3.2 Admin Access Policy

- **Target Role:** `authenticated` (Logged-in Staff).
- **Permitted Operation:** `ALL` (`SELECT`, `UPDATE`, `DELETE`).
- **Authentication:** Requires a valid JWT (JSON Web Token) obtained via Email/Password login on `admin.html`.

3.4 Serverless Edge Functions (`index.ts`)

Function Name: `resend-email`

Runtime: Deno

Purpose: Handles transactional email notifications via the Resend API.

3.4.1 The Event-Driven Workflow

1. **Trigger:** I configured a **Database Webhook** to watch for `INSERT` events on the `appointments` and `inquiries` tables.
2. **Execution:** The payload (the new database row) is sent to the Edge Function.
3. **Logic (`index.ts`):**
   - **Validation:** The script first confirms the event type is `INSERT`. If it is an update or delete, the function exits early to save resources.
   - **Template Selection:** It inspects `payload.table`.
     - *If Appointment:* It generates a rich HTML table containing the patient's name, requested date, and a "Click to Call" button (`<a href="tel:...">`) for immediate action.
     - *If Inquiry:* It generates a simplified notification card.

- **Dispatch:** It sends a POST request to `https://api.resend.com/emails` using the secure API Key retrieved from the environment.

3.5 Infrastructure as Code (Configuration)

I defined the backend environment strictly in code files to ensure reproducibility and security.

3.5.1 Project Configuration (`config.toml`)

- **Project Identity:** I defined the project as `DrGhoshBackend`.
- **Function Security:** I set `verify_jwt = true`. This is a critical security measure I enforced to ensure the Edge Function rejects any request that does not come from a trusted Supabase source (i.e., the database webhook). External hackers cannot trigger the function via `curl` or Postman.
- **Local Ports:** I configured the API Gateway on port `54321` and the Database on `54322` for local testing.

3.5.2 Dependency Management (`deno.json`)

Instead of a massive `node_modules` folder which can be fragile, I used a **Deno Import Map**.

- **Mapping:** `"@supabase/functions-js": "jsr:@supabase/functions-js@^2"`
- **Benefit:** This allows the function to import Supabase tools cleanly via URLs without complex build steps, reducing the deployment size significantly.

3.6 Deployment & DevOps Workflow (PowerShell)

To deploy this fragmented architecture, I utilized the Supabase CLI via PowerShell. These are the specific commands used to operationalize the backend:

Step 1: Environment Initialization

I used `supabase init` to create the scaffolding and the `config.toml` file, establishing the link between my local machine and the cloud project.

Step 2: Secrets Management

I ensured API keys were never hardcoded in the `index.ts` file. Instead, I injected them into the Supabase Vault using:

```
supabase secrets set RESEND_API_KEY=re_123456789...
```

The code retrieves this securely at runtime via `Deno.env.get('RESEND_API_KEY')`.

Step 3: Deploying the Function

I uploaded the logic to the global edge network using:

```
supabase functions deploy resend-email
```

Note: This command reads the `config.toml` to determine the correct entry point (`./functions/resend-email/index.ts`).

3.7 Storage Architecture

Bucket Name: `uploads`

Content: Patient medical reports (PDF, JPG, PNG).

Directory Structure: Flat (Root Level).

Access Policy:

I allowed Public `INSERT` and `SELECT` access.

Reasoning: The file URL (`file_url`) is generated upon upload and included in the email notification. For the doctor to view the file immediately without logging into a separate system, the specific file link must be publicly resolvable. I maintained security by ensuring filenames are obfuscated using UUIDs (Universally Unique Identifiers) so they cannot be guessed.

4. Conclusion & Future Roadmap

4.1 Project Conclusion

This project successfully establishes a robust digital infrastructure for Dr. Jay Deep Ghosh, transforming how patients in the Northeast region access specialized orthopaedic care. By moving away from generic hospital directories to a custom, brand-centric platform, we have achieved the dual goals of **Authority Building** and **Operational Efficiency**.

The decision to utilize a **Serverless Architecture (Supabase + GitHub Pages)** has proven to be highly effective. It eliminates the recurring costs and maintenance headaches associated with traditional server management while providing enterprise-grade security for patient data. The site is fast, mobile-optimized, and compliant with modern web standards, ensuring it serves as a reliable asset for the client's practice for years to come.

4.2 Future Upgrade Plans

To ensure the platform evolves with the growing needs of the clinic, I have outlined a strategic roadmap for the next phase of development. These upgrades focus on automation, real-time data, and enhanced user experience.

Phase 1: Content & Operational Refinements

- **Dynamic Content Management:** I plan to implement a lightweight Content Management System (CMS) or admin interface that allows the clinic staff to add, remove, or polish text and images on the "Procedures" and "Testimonials" pages without needing to touch the raw HTML code.
- **Automated Patient Confirmation:** Currently, the system notifies the doctor of a new request. The next iteration will trigger an **automated confirmation email** sent directly to the patient once the status is changed to "Confirmed" in the Admin Dashboard. This closes the communication loop instantly.

Phase 2: Advanced Scheduling Logic

- **Real-Time Availability Calendar:** I intend to upgrade the current date picker to a **Dynamic Calendar**. This will query the database in real-time to show available vs. booked slots, preventing double-booking and reducing the back-and-forth communication required to finalize a time.
- **Capacity Management:** To prevent burnout and ensure quality care, I will add logic to the Admin Panel that allows Dr. Ghosh to set a **fixed cap on appointments per day** (e.g., maximum 10 slots). Once this limit is reached, that specific date will automatically disable on the frontend booking form.

Phase 3: UI/UX & Design Enhancements

- **Dark Mode Support:** Given the growing preference for dark interfaces, especially for users browsing at night, I plan to utilize the existing CSS variable system to implement a system-aware Dark Mode.
- **Skeleton Loading Screens:** To further perceive performance, I will replace the current loading spinners with "Skeleton Screens" (gray placeholders that mimic the content shape). This provides a smoother visual transition while data is being fetched from Supabase.
- **Multi-Language Support:** Considering the linguistic diversity of patients in Assam and the Northeast, I propose adding a toggle for a localized version of the site (Assamese/Hindi) to make high-quality medical information accessible to a wider demographic.